

自動化排班系統

Auto Scheduling System

Product Requirements Document (PRD)

產品設計規格文件 — TA 討論

114-2 資料庫管理 期末專案

第 21 組

版本 2.0 | 2026 年 4 月 10 日 | MySQL 版

技術架構: React + Express.js + MySQL

一、System Overview

1.1 專案背景

台灣服務業與校園兼職普遍以人工方式排班，管理者需逐一彙集員工課表、透過 LINE 群組記事本劃假，每週耗費約 2 小時。本專案源自團隊三位組員在星巴克、補習班、政大 EMBA 的實際打工經驗。

1.2 訪談摘要

實際訪談星巴克政大門市店經理，每週花費 2 小時排班，每月隱性管理成本約 1,600 元（時薪 200 元計）。願意支付每月最多 500 元使用自動排班系統。

1.3 核心痛點

- 逐一規劃課表與劃假，效率低落
- 課外活動佔去學生大量時間，排班衝突難以掌握
- 缺乏透明換班機制，臨時調班溝通成本高
- 早中晚三班人力配置 2-1-2，遇特殊活動需彈性調整

1.4 解決方案總覽

1. 行程輸入：員工上傳 CSV 課表 + 勾選可用時段矩陣
2. 智慧排班：系統交叉比對 Availability 與 CourseSchedule，自動產出班表
3. 換班申請：員工發起換班請求，系統自動列出可換班人選
4. 薪資自動化：班次結束後自動產生 PayRecord

1.5 市場現有產品比較

市場上現有 104 排班系統、Mayohr、Nueip 等，平均定價約 30-70 元/人/月。本系統差異化功能：

- 學生課表 CSV 匯入整合
- 員工自助換班請求與即時通知
- 技能層級管理（新人避讓 / 老手比例控制）
- 薪資自動結算與報表
- Line 通知

二、System Features

將每個功能拆解至具體的使用者操作層級，並標註對應的 SQL 操作與資料表。

Feature F1: 員工帳號管理 (Employee Management)

- 管理者可以新增員工帳號，設定姓名與初始技能等級 (level: junior/senior)
- 管理者可以查看所有員工清單，包含姓名與技能等級
- 管理者可以修改員工的技能等級 (junior ↔ senior)
- 管理者可以刪除員工帳號 (連帶刪除該員工的課表、可用時段等關聯資料)
- 員工可以查看自己的個人資訊

Feature F2: 課表匯入與管理 (Course Schedule Management)

- 員工可以上傳 CSV 檔案匯入固定課表 (批次 INSERT 至 CourseSchedule)
- 系統在匯入前顯示預覽畫面，讓員工確認課表內容
- 員工可以手動新增單筆課表時段 (指定 day_of_week, start_time, end_time, CourseName)
- 員工可以刪除已匯入的課表紀錄
- 員工可以重新匯入課表 (先 DELETE 舊資料再 INSERT 新資料)
- 系統在排班時自動讀取 CourseSchedule 排除不可用時段

Feature F3: 可用時段設定 (Availability Management)

- 使用者可透過類似 When2Meet 的互動式週時間表選擇可上班時段。
- 系統支援點擊與拖曳操作，快速標記可用與不可用時間。
- 使用者可匯入個人課表，系統自動將上課時段標記為不可用。
- 所有可用時間資料會即時儲存至資料庫，並允許使用者隨時修改。
- 系統將勾選結果以 UPSERT 方式寫入 Availability 資料表
- 系統在排班時讀取 Availability 交叉比對 CourseSchedule，計算實際可排班時段

選擇可上班時間

點擊或拖曳選擇即可上班的時段

日期範圍: 4/13 - 4/19 (2026)

☒ 可上班 ☐ 不可上班

	週一 4/13	週二 4/14	週三 4/15	週四 4/16	週五 4/17	週六 4/18	週日 4/19
8:00							
9:00							
10:00							
11:00							
12:00							
13:00							
14:00							
15:00							
16:00							
17:00							
18:00							
19:00							
20:00							
21:00							

已選擇時段: 16 小時

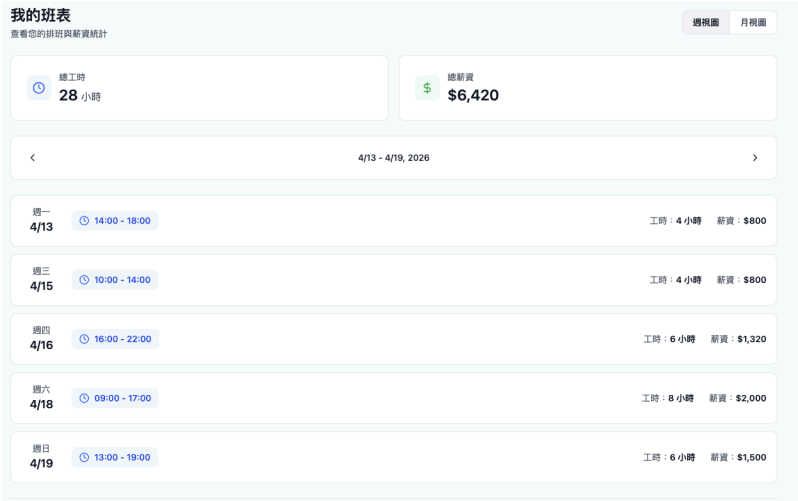
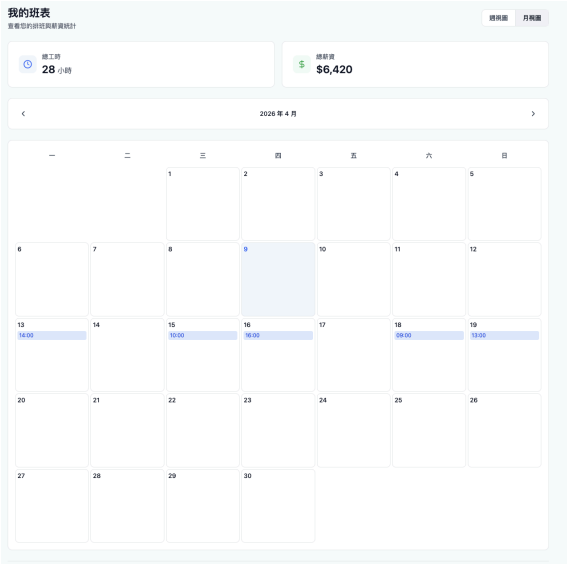
儲存可用時段

Feature F4: 智慧排班 (Auto Scheduling)

- 系統根據所有使用者的可用時間自動產生排班結果。
- 排班僅會安排於使用者標記為可用的時段。
- 系統可依需求設定各時段所需人數，確保人力配置合理。
- 管理者可以定義班次(設定 schedule_date, start_time, end_time, Employee_num)
- 管理者可以觸發自動排班，系統執行 SELECT JOIN 查詢所有員工的可用時段
- 排班引擎交叉比對 Availability 與 CourseSchedule，排除衝突時段
- 排班引擎依據 Employee.level 確保每班至少有一位 senior 員工
- 排班結果自動 INSERT 至 WorkSchedule，並在 ScheduleSource 標記為 'auto'
- 管理者可以在自動排班後手動微調班表(UPDATE WorkSchedule)，ScheduleSource 標記為 'manual'

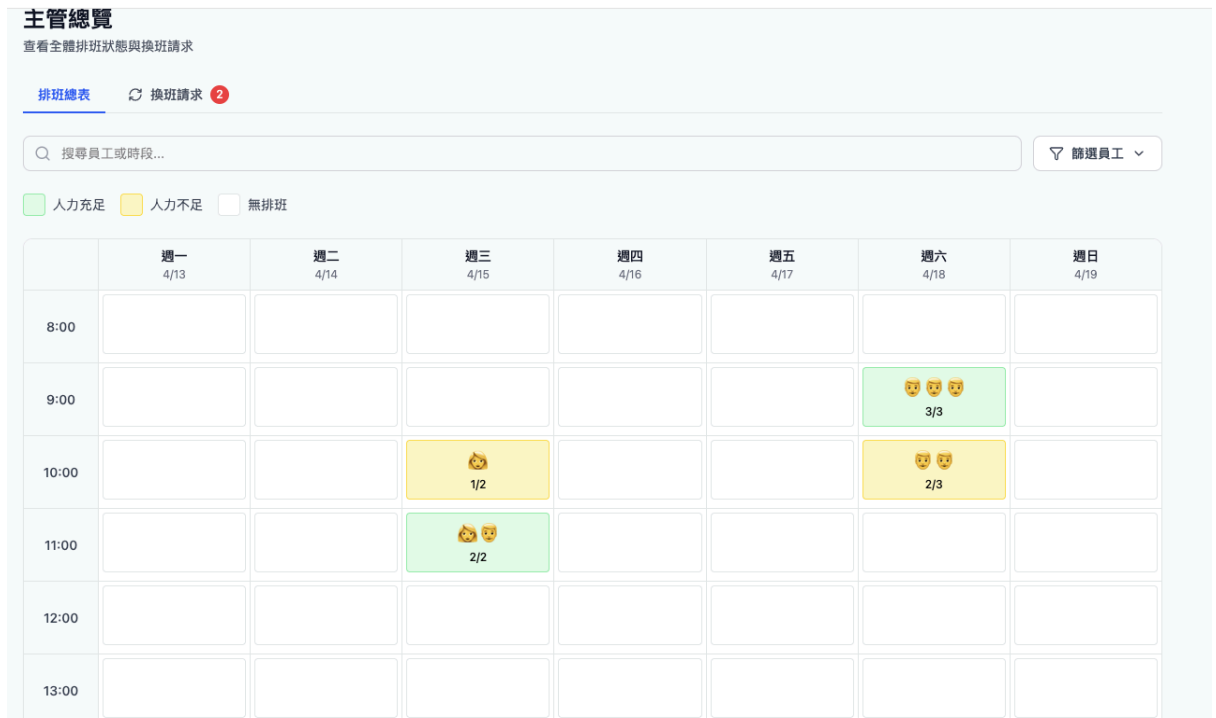
Feature F5: 班表查詢與可視化 (Schedule Viewing)

- 使用者可查看個人排班結果(支援週視圖與月視圖切換) (SELECT WorkSchedule JOIN Employee)
- 系統提供總工時統計功能。
- 系統依據工時計算並顯示總薪資。
- 員工可以查看自己被排定的班次
- 系統以紅色標示人力不足的班次(Employee_num > 實際排班人數)
- 系統顯示每個班次的排班來源(auto / manual / swap)



Feature : 主管排班總覽

- 主管可查看所有員工的排班總表(以時間網格呈現)。
- 每個時段會顯示已排班人員名單。
- 系統以視覺化方式呈現人力狀態(如:人力充足、不足或無排班)
- 提供員工篩選功能, 方便查詢特定人員排班情況。



Feature F6: 換班申請 (Swap Request)

- 員工可以選擇自己的某個班次, 發起換班申請
- 主管需進行最終審核。
- 經雙方與主管同意後, 系統將自動更新排班結果。
- 系統提供換班狀態追蹤(如: 等待中、已同意、已核准、已拒絕)。
- 系統自動查詢該時段 Availability 為 available 的其他員工名單
- 員工選擇換班對象後, 填寫原因, 送出 swapRequest(INSERT)
- 被指定的接收者可以查看收到的換班請求列表
- 接收者可以同意或拒絕換班(UPDATE swapRequest.status)
- 同意換班後, 系統自動更新 WorkSchedule 的人員配置, ScheduleSource 標記為 'swap'
- 員工可以查看自己發出的換班請求歷史與狀態

換班申請

管理您的換班請求與狀態

我的班次

週一 4/13	🕒 14:00 - 18:00	已申請換班
週三 4/15	🕒 10:00 - 14:00	申請換班
週四 4/16	🕒 16:00 - 22:00	申請換班
週六 4/18	🕒 09:00 - 17:00	申請換班

換班請求狀態

週一 4/13	🕒 14:00 - 18:00	⚠️ 等待回應
👤 換班對象：李美麗		

收到的換班請求 2

 李美麗 想與您交換班次 4/9 週三 4/15	🕒 10:00 - 14:00	<input checked="" type="button" value="接受"/> 拒絕
 張大衛 想與您交換班次 4/9 週六 4/18	🕒 09:00 - 17:00	<input checked="" type="button" value="接受"/> 拒絕

Feature F7: 薪資紀錄 (Pay Record)

- 班次結束後，系統為該班次中每位員工產生一筆 PayRecord (INSERT)
- PayRecord 記錄 wage, work_hour, wage_per_hour, 透過 FK 關聯 WorkSchedule 與 Employee
- 員工可以查看自己的薪資紀錄 (SELECT PayRecord WHERE Employee_ID = ?)
- 管理者可以查看所有員工的薪資紀錄
- 支援按月/按週篩選薪資紀錄 (WHERE schedule_date BETWEEN ? AND ?)
- 系統可產出薪資統計報表 (SELECT SUM(wage), SUM(work_hour) GROUP BY Employee_ID)

DEMO前端UI版本

<https://www.figma.com/make/JRL6PVCDKMNnxkeju89eBb/Shift-Scheduling-System-UI?t=uX8lwKr5jgsy5giy-1>

<https://scheduling-system.zeabur.app/login>
<https://shiftsched-u5kwqxgq.manus.space/>

<https://www.figma.com/make/U0BHNGpBVtuuR4whtEzRw4/Enterprise-Scheduling-System-Interface?t=IAv8mGo2g7ivZaVd-1>

三、ERD 資料模型

3.1 實體 (Entity) 清單

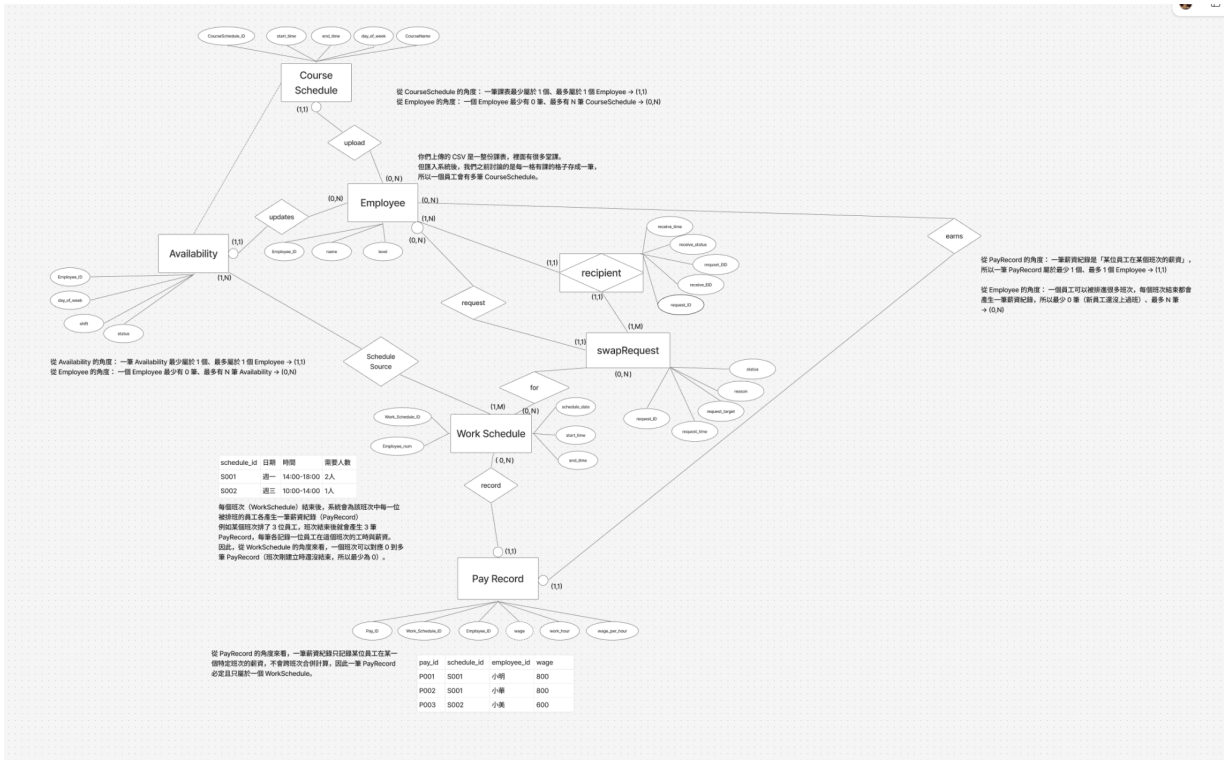
實體	屬性(PK/FK 標示)	說明
Employee	employee_id (PK), name, level	系統使用者(工讀生/員工), 儲存基本資訊與技能等級
CourseSchedule	CourseSchedule_ID (PK), employee_id (FK), start_time, end_time, day_of_week, CourseName	員工固定課表, CSV 匯入後每格有課的時段存為一筆
Availability	Employee_ID (FK/PK), day_of_week (PK), shift (PK), status	員工每週可用時段矩陣, 複合主鍵
WorkSchedule	Work_Schedule_ID (PK), Employee_num, schedule_date, start_time, end_time	實際排定的工作班表
PayRecord	Pay_ID (PK), Work_Schedule_ID (FK), Employee_ID (FK), wage, work_hour, wage_per_hour	薪資紀錄, 班次結束後為每位員工產生一筆
swapRequest	request_ID (PK), request_EID (FK), receive_EID (FK), request_target, request_time, reason, status	換班申請, 雙向關聯 Employee
ScheduleSource	Work_Schedule_ID (FK), source_type	排班來源追溯(自動/手動/換班)

3.2 關係 (Relationship) 清單

關係	連結 & 基數	語義
upload	Employee (1,1) ↔ CourseSchedule (0,N)	一筆課表屬於 1 位員工; 1 位員工可有 0~N 筆課表
updates	Employee (1,1) ↔ Availability (0,N)	一筆可用時段屬於 1 位員工; 1 位員工可有 0~N 筆紀錄
request	Employee (1,1) ↔ swapRequest (0,N)	申請者: 1 筆換班由 1 位員工發起
recipient	Employee (1,1) ↔ swapRequest (0,N)	接收者: 1 筆換班有 1 位接收者
for	swapRequest (0,N) ↔ WorkSchedule (1,M)	換班針對特定班次
record	WorkSchedule (0,N) ↔ PayRecord (1,1)	班次結束產生薪資紀錄
earns	Employee (0,N) ↔ PayRecord (1,1)	員工獲得薪資紀錄

3.3 ERD 設計要點

<https://www.figma.com/board/JJQaUFMdCRoXnve03sYWJ/%E8%B3%87%E6%96%99%E5%BA%AB%E7%AE%A1%E7%90%86?node-id=0-1&t=KJeySQ9c2Oo1itv4-1>



Availability 獨立成實體

將員工可用時段從 Employee 抽出為獨立實體，以 (Employee_ID, day_of_week, shift) 為複合主鍵。排班引擎可直接查詢可排班矩陣，無須解析課表後再推導。

swapRequest 雙向關聯

swapRequest 透過 request 和 recipient 兩條關係連結 Employee, 分別記錄申請者 (request_EID) 與接收者 (receive_EID)。搭配 status 欄位完整追蹤換班流程。

PayRecord 按班次產生

每個班次結束後, 系統為該班次中每位員工各產生一筆 PayRecord。同時關聯 WorkSchedule (1,1) 和 Employee (1,1), 確保薪資紀錄的完整性。

四、Relational Schema & BCNF 驗證

4.1 Relational Schema

Employee (employee_id, name, level)

CourseSchedule (CourseSchedule_ID, employee_id*, start_time, end_time, day_of_week, CourseName)

Availability (Employee_ID*, day_of_week, shift, status) — PK: (Employee_ID, day_of_week, shift)

WorkSchedule (Work_Schedule_ID, Employee_num, schedule_date, start_time, end_time)

PayRecord (Pay_ID, Work_Schedule_ID*, Employee_ID*, wage, work_hour, wage_per_hour)

swapRequest (request_ID, request_EID*, receive_EID*, request_target*, request_time, reason, status)

ScheduleSource (Work_Schedule_ID*, source_type)

* 表示外鍵 (Foreign Key)

4.2 BCNF 驗證

以下驗證每張資料表的所有非平凡 (non-trivial) 函數相依, 其左側皆為 superkey, 因此滿足 BCNF:

資料表	函數相依 (FD)	BCNF 驗證
Employee	employee_id → name, level	employee_id 是 CK, 左側為 superkey ✓
CourseSchedule	CourseSchedule_ID → employee_id, start_time, end_time, day_of_week, CourseName	CourseSchedule_ID 是 CK ✓
Availability	(Employee_ID, day_of_week, shift) → status	複合 CK 決定 status ✓
WorkSchedule	Work_Schedule_ID → Employee_num, schedule_date, start_time, end_time	Work_Schedule_ID 是 CK ✓
PayRecord	Pay_ID → Work_Schedule_ID, Employee_ID, wage, work_hour, wage_per_hour	Pay_ID 是 CK ✓
swapRequest	request_ID → request_EID, receive_EID, request_target, request_time, reason, status	request_ID 是 CK ✓
ScheduleSource	Work_Schedule_ID → source_type	Work_Schedule_ID 是 CK ✓

結論: 所有 7 張資料表均滿足 BCNF, 無需進一步分解。

五、SQL 實作規劃

5.1 SQL 操作對照表

以下列出每個功能對應的 SQL 操作，確保 Main Functions Demo 能展示系統與 MySQL 的實際連結：

SQL 操作	涉及資料表	對應功能說明
CREATE	Employee, CourseSchedule, Availability, WorkSchedule, PayRecord, swapRequest, ScheduleSource	建立 7 張資料表，含 PK、FK、CONSTRAINT
INSERT	Employee	管理者新增員工 / 員工自行註冊
INSERT	CourseSchedule	CSV 匯入課表，批次 INSERT
INSERT / UPDATE	Availability	員工更新可用時段矩陣(UPSERT)
SELECT + JOIN	Availability JOIN CourseSchedule	排班引擎：交叉比對可用時段與課表衝突
INSERT	WorkSchedule	排班結果寫入班表
SELECT	WorkSchedule JOIN Employee	班表查詢(週視圖/月視圖)
INSERT	swapRequest	員工發起換班申請
UPDATE	swapRequest	接收者同意/拒絕換班(更新 status)
UPDATE	WorkSchedule	換班核准後更新班表人員
INSERT	PayRecord	班次結束後產生薪資紀錄(JOIN WorkSchedule + Employee)
SELECT	PayRecord JOIN Employee	薪資查詢(按月/週篩選)
SELECT + GROUP BY	PayRecord	薪資統計報表(月總工時、月總薪資)
DELETE	CourseSchedule	員工移除舊課表後重新匯入

5.2 MySQL DDL (建表語句)

以下為 7 張資料表的 CREATE TABLE 語句，包含 PK、FK、CONSTRAINT：

```
-- Employee
CREATE TABLE Employee (
  employee_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL,
  level ENUM('junior', 'senior') DEFAULT 'junior'
);

-- CourseSchedule
```

```
CREATE TABLE CourseSchedule (
  CourseSchedule_ID INT PRIMARY KEY AUTO_INCREMENT,
  employee_id INT NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  day_of_week ENUM('Mon','Tue','Wed','Thu','Fri','Sat','Sun') NOT NULL,
  CourseName VARCHAR(100),
  FOREIGN KEY (employee_id) REFERENCES Employee(employee_id)
  ON DELETE CASCADE
);
```

-- Availability

```
CREATE TABLE Availability (
  Employee_ID INT NOT NULL,
  day_of_week ENUM('Mon','Tue','Wed','Thu','Fri','Sat','Sun') NOT NULL,
  shift ENUM('morning','afternoon','evening') NOT NULL,
  status ENUM('available','unavailable') DEFAULT 'available',
  PRIMARY KEY (Employee_ID, day_of_week, shift),
  FOREIGN KEY (Employee_ID) REFERENCES Employee(employee_id)
  ON DELETE CASCADE
);
```

-- WorkSchedule

```
CREATE TABLE WorkSchedule (
  Work_Schedule_ID INT PRIMARY KEY AUTO_INCREMENT,
  Employee_num INT NOT NULL DEFAULT 1,
  schedule_date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL
);
```

-- PayRecord

```
CREATE TABLE PayRecord (
  Pay_ID INT PRIMARY KEY AUTO_INCREMENT,
  Work_Schedule_ID INT NOT NULL,
  Employee_ID INT NOT NULL,
  wage DECIMAL(10,2) NOT NULL,
  work_hour DECIMAL(5,2) NOT NULL,
  wage_per_hour DECIMAL(8,2) NOT NULL,
  FOREIGN KEY (Work_Schedule_ID) REFERENCES WorkSchedule(Work_Schedule_ID),
  FOREIGN KEY (Employee_ID) REFERENCES Employee(employee_id)
);
```

-- swapRequest

```
CREATE TABLE swapRequest (
  request_ID INT PRIMARY KEY AUTO_INCREMENT,
  request_EID INT NOT NULL,
  receive_EID INT NOT NULL,
  request_target INT NOT NULL,
  request_time DATETIME DEFAULT CURRENT_TIMESTAMP,
  reason TEXT,
  status ENUM('pending','approved','rejected') DEFAULT 'pending',
  FOREIGN KEY (request_EID) REFERENCES Employee(employee_id),
  FOREIGN KEY (receive_EID) REFERENCES Employee(employee_id),
  FOREIGN KEY (request_target) REFERENCES WorkSchedule(Work_Schedule_ID)
);
```

-- ScheduleSource

```
CREATE TABLE ScheduleSource (
  Work_Schedule_ID INT PRIMARY KEY,
  source_type ENUM('auto','manual','swap') NOT NULL,
  FOREIGN KEY (Work_Schedule_ID) REFERENCES WorkSchedule(Work_Schedule_ID)
);
```

六、前端頁面規劃

#	頁面	角色	核心功能	優先
P1	登入 / 註冊頁	員工 & 管理者	帳號登入、角色選擇、新用戶註冊	高
P2	管理者 Dashboard	管理者	本週班表總覽、人力缺口警告、待處理換班數	高
P3	員工 Dashboard	員工	我的班表、換班狀態、薪資摘要	高
P4	員工管理頁	管理者	員工 CRUD、等級設定	高
P5	課表匯入頁	員工	CSV 上傳、手動編輯、匯入預覽	高
P6	可用時段設定頁	員工	7 天 × 班次矩陣勾選	高
P7	班表管理頁(週/月視圖)	管理者	自動排班觸發、手動微調、衝突標示	高
P8	排班規則設定頁	管理者	班次定義、人數、新人上限	中
P9	換班申請頁	員工	選班次 → 系統列可換人 → 送出	高
P10	換班審核頁	管理者/員工	同意/拒絕、歷史紀錄	中
P11	薪資紀錄頁	全角色	按月/週查看明細、匯出	中

6.1 頁面流程簡述

管理者流程: 登入 → Dashboard → 員工管理 / 班表管理 / 排班規則設定 / 換班審核 / 薪資紀錄

員工流程: 登入 → Dashboard → 課表匯入 / 可用時段設定 / 查看班表 / 換班申請 / 薪資查詢

七、技術架構

層級	技術	理由
前端	React 18 + Vite + Tailwind CSS	元件化架構、快速原型
後端	Node.js + Express.js	課程所學、JS 全端
資料庫	MySQL 8.0	Relational DB, 課程要求, SQL 實作
ORM / Query	mysql2 (raw SQL)	直接撰寫 SQL, 符合 Demo 評分要求
認證	JWT + bcrypt	無狀態認證, 前後端分離
部署	Vercel (前端) + Railway (後端 + MySQL)	免費額度, 適合學生專案

7.1 MySQL 資料庫

- 使用 Relational Database 並以 SQL commands 實作
- 資料結構明確: ERD 中 7 個實體的關聯明確, 適合 RDBMS
- BCNF 驗證: 所有資料表均滿足 BCNF, 無冗餘
- JOIN 查詢需求: 排班引擎需要大量交叉比對, SQL JOIN 效率優於 NoSQL

7.2 專案資料夾結構

client/ (React 前端)

- src/pages/ — 對應 11 個頁面
- src/components/ — 可重用元件
- src/services/ — API 呼叫封裝

server/ (Express 後端)

- routes/ — API 路由
- controllers/ — 業務邏輯
- models/ — SQL 查詢封裝
- db/ — MySQL 連線設定 + DDL 腳本

八、Implementation Plan

8.1 開發排程 (Gantt Chart 簡版)

Phase 1 — 基礎建設 (Week 1-2)

1. React + Vite + Tailwind + Express + MySQL 環境搭建
2. MySQL DDL 執行: 建立 7 張資料表
3. JWT 認證系統 + 角色權限中介層
4. 員工管理 CRUD (F1)

Phase 2 — 核心功能 (Week 3-4)

1. CSV 課表匯入 (F2)
2. 可用時段設定 (F3)
3. 班表管理 + 自動排班引擎 (F4, F5)

Phase 3 — 進階功能 (Week 5-6)

1. 換班申請完整流程 (F6)
2. 薪資結算 + 報表 (F7)
3. UI 打磨、回應式設計、測試

8.2 失敗回復計畫

- 資料庫備份: MySQL 定期 mysqldump, 存放於獨立儲存空間
- 交易安全: 換班操作使用 MySQL Transaction (BEGIN / COMMIT / ROLLBACK) 確保一致性
- 排班引擎失敗: 產出警告而非強制寫入, 管理者可手動介入

8.3 Concurrency Control

- 換班衝突: 同一班次同時有多筆換班申請時, 使用 SELECT ... FOR UPDATE 鎖定該班次紀錄
- 自動排班: 排班過程中鎖定 WorkSchedule 資料表, 避免同時手動調整
- Isolation Level: 使用 MySQL 預設的 REPEATABLE READ

8.4 安全控制

- 認證: JWT Token 有效期 24 小時, 密碼 bcrypt 雜湊儲存
- 權限控制: 中介層檢查 role (admin/employee), 管理者專屬 API 僅限 admin 存取
- SQL Injection 防護: 使用 mysql2 的 Prepared Statements, 所有查詢參數化

8.5 Agile 實踐 (Bonus)

- 採用 Kanban 看板管理開發進度 (Notion / Trello)
- 每週 Sprint Review: 組員展示已完成功能, 調整下週優先級
- 提供 Kanban 看板截圖作為證據